

SUPPLY MANAGEMENT SYSTEM

Functional Specification Document

Addendum 29 - SCM Commercial Enablement

Business Partners | Sales Orders | Fulfillment | Invoicing | Customer Finance

Implementation Specification for Claude Code

Version 1.3 - Logistics-Aligned (Delivery Orders + IStockReservationService)

Property	Value
Document ID	SMS-FSD-ADD-029
Version	1.3
Date	2026-09-19
Classification	Internal - Development
Architecture	.NET 8 / EF Core 8 / SQL Server 2022 / React 18
Status	Ready for Implementation

Table of Contents

- Section 0 - Claude Code Implementation Instructions
- Section 1 - Business Partner Entity (Expanded Partner Types)
- Section 2 - Product Pricing Enhancement & Default Supplier
- Section 3 - Sale Order Administration & Configuration
- Section 4 - Sale Orders (3-Scenario Availability Check)
- Section 5 - Sale Order Email Intimation System
- Section 6 - Back-to-Back PO (Draft + Approval Control)
- Section 7 - Fulfillment via Existing Delivery Orders
- Section 8 - Self-Pickup Flow (via Delivery Order + Goods Issue)
- Section 9 - Sales Invoicing & Customer Payments
- Section 10 - Customer Ledger (Receivables)
- Section 11 - Product Ledger
- Section 12 - Inventory Impact (Sales Side)
- Section 13 - PO-SO Linkage via trace_id / DocumentTimeline
- Section 14 - Procurement Impact (Back-to-Back & Drop Ship)
- Section 15 - Sales Reports
- Section 16 - Implementation Order (30 Dev Days)
- Section 17 - Acceptance Criteria & Test Scenarios

Section 0 - Claude Code Implementation Instructions

This document is structured as an implementation specification for Claude Code. Before writing any code, the agent **MUST**:

- Inspect the existing codebase to understand current entity structure, naming conventions, and patterns
- Follow existing repository/service/controller patterns exactly - do NOT invent new architectural patterns
- Use the existing EF Core DbContext and migration pipeline (dotnet ef migrations add)
- Respect the existing multi-tenancy pattern (org_id discriminator with HasQueryFilter)
- Follow existing schema isolation: auth, lookups, suppliers, inventory, demand, procurement, warehouse, logistics, finance, reports, workflow, hangfire, material, tenant
- Use AutoMapper for DTOs, FluentValidation for input validation, MediatR for domain events
- Use QuestPDF for any new PDF generation
- All background jobs via Hangfire (hangfire schema)
- Check existing Shipment/Delivery implementation before building Fulfillment - reuse patterns and structures
- USE the existing logistics.delivery_orders infrastructure with from_source_type = SALE_ORDER for sales fulfillment. Do NOT create separate warehouse.Fulfillments — the delivery_orders 15-status state machine already handles DRAFT→PICKING→PACKED→GOODS_ISSUED→DELIVERED
- USE the existing IStockReservationService (SMS.Shared) with ReservationSourceType.SALES_ORDER for sales stock reservation. Do NOT create a separate inventory.StockReservations table — the shared service handles MIR, DELIVERY, and SALES_ORDER sources
- USE existing pick_lists, shipment_packages, consignments, goods issue patterns from the logistics module. These are already built and tested (1531 logistics tests).
- USE existing logistics.addresses table for structured shipping addresses instead of NVARCHAR(500) flat strings
- USE existing logistics.document_number_sequences with RowVersion concurrency for all new document number generation
- REFERENCE existing permissions: DELIVERY_VIEW, DELIVERY_EDIT, PICKING, PACKING, DISPATCH, GOODS_ISSUE, SHIPMENT_BOOK, POD_CAPTURE
- Check existing Supplier Scorecard implementation (Addendum 23) for best-match supplier logic
- Check existing Supplier Rate Card implementation (Addendum 28) for latest price resolution
- Reuse existing trace_id / DocumentTimelines pattern (Quotation→PO, PR→PO) for SO→PO linkage. SAME propagation, SAME JSON events array, SAME Hangfire non-blocking write
- MULTI-TENANCY: Every new table MUST include org_id column with EF Core HasQueryFilter. All queries filter by org_id from JWT claim. Hangfire jobs pass org_id as explicit parameter (no HttpContext in background). This is NON-NEGOTIABLE across all 14 schemas.

Implementation order is defined in Section 15. Each section includes exact table schemas, API endpoints, business rules, and test scenarios.

Section 1 - Business Partner Entity (Expanded Partner Types)

1.1 Migration Strategy: Rename, Not Copy

The existing Suppliers table is renamed to BusinessPartners. This is a zero-data-loss migration. All existing FK references (PurchaseOrders, GRNs, SupplierLedger, etc.) are updated to point to the renamed table. No data is duplicated.

- Step 1: Rename table [suppliers.Suppliers] to [suppliers.BusinessPartners]
- Step 2: Add new columns (partner_type, customer fields) with defaults
- Step 3: UPDATE all existing rows SET partner_type = 'VENDOR'
- Step 4: Rename FK columns across referencing tables (supplier_id -> partner_id) via sp_rename
- Step 5: Update EF Core entity from Supplier to BusinessPartner
- Step 6: Update all repository/service/controller references
- Step 7: Add backward-compatible API aliases (/api/suppliers -> /api/partners?type=VENDOR)

1.2 Partner Type Enum (Expanded)

The partner_type field supports the following values:

partner_type	Description	Can be Vendor?	Can be Customer?	Can be Carrier?	Can be Service Provider?
VENDOR	Goods supplier	Yes	No	No	No
CUSTOMER	Buyer/client	No	Yes	No	No
CARRIER	Shipping/logistics carrier	No	No	Yes	No
SERVICE_PROVIDER	Services (maintenance, consulting, etc.)	No	No	No	Yes
BOTH	Vendor + Customer (trades both ways)	Yes	Yes	No	No
VENDOR_CARRIER	Supplier who also handles shipping	Yes	No	Yes	No
VENDOR_SERVICE	Supplier who also provides services	Yes	No	No	Yes
FULL_PARTNER	All capabilities	Yes	Yes	Yes	Yes

Implementation approach: Store partner_type as NVARCHAR(20). Additionally, store four boolean flags for flexible querying:

Partner Type Flags (added to BusinessPartners)

Column	Type	Nullable	Description
is_vendor	BIT	NOT NULL	Can supply goods (default 0)
is_customer	BIT	NOT NULL	Can purchase from us (default 0)
is_carrier	BIT	NOT NULL	Can handle shipping/logistics (default 0)
is_service_provider	BIT	NOT NULL	Can provide services (default 0)

The partner_type enum value is derived from the combination of flags. When saving, the system auto-computes partner_type from the flags. This allows flexible querying (WHERE is_carrier = 1) while maintaining a readable type label.

1.3 BusinessPartners Table Schema

suppliers.BusinessPartners

Column	Type	Nullable	Description
partner_id	BIGINT IDENTITY	NOT NULL	PK (was supplier_id)
org_id	INT	NOT NULL	Tenant discriminator
partner_code	NVARCHAR(30)	NOT NULL	Auto-generated (was supplier_code)
partner_type	NVARCHAR(20)	NOT NULL	VENDOR CUSTOMER CARRIER SERVICE_PROVIDER BOTH VENDOR_CARRIER VENDOR_SERVICE FULL_PARTNER
is_vendor	BIT	NOT NULL	Vendor capability flag
is_customer	BIT	NOT NULL	Customer capability flag
is_carrier	BIT	NOT NULL	Carrier capability flag
is_service_provider	BIT	NOT NULL	Service provider capability flag
company_name	NVARCHAR(200)	NOT NULL	Legal entity name
trade_name	NVARCHAR(200)	NUL L	Trading-as name
contact_person	NVARCHAR(100)	NUL L	Primary contact
email	NVARCHAR(150)	NUL L	Primary email
phone	NVARCHAR(30)	NUL L	Primary phone
address_line 1	NVARCHAR(200)	NUL L	Street address
address_line 2	NVARCHAR(200)	NUL L	Additional address
city	NVARCHAR(100)	NUL L	City
state_province	NVARCHAR(100)	NUL L	State/Province
country	NVARCHAR(100)	NUL L	Country
postal_code	NVARCHAR(20)	NUL L	ZIP/Postal
tax_id	NVARCHAR(50)	NUL L	VAT/Tax registration
credit_limit	DECIMAL(18,2)	NUL L	Customer credit limit
payment_terms_days	INT	NOT NULL	Net payment days (default 30)
currency_code	NVARCHAR(3)	NOT NULL	Default currency (ISO 4217)
bank_name	NVARCHAR(100)	NUL L	Bank for payments

Column	Type	Nullable	Description
bank_account	NVARCHAR(50)	NULL	Bank account number
iban	NVARCHAR(50)	NULL	IBAN
swift_code	NVARCHAR(20)	NULL	SWIFT/BIC
vehicle_types	NVARCHAR(500)	NULL	Carrier: types of vehicles available
service_categories	NVARCHAR(500)	NULL	Service Provider: categories of services offered
is_active	BIT	NOT NULL	Soft active flag
rating	NVARCHAR(2)	NULL	A/B/C/D/F scorecard grade
notes	NVARCHAR(MAX)	NULL	Free-text notes
created_by	NVARCHAR(100)	NOT NULL	Creator user
created_at	DATETIME2	NOT NULL	Creation timestamp
updated_by	NVARCHAR(100)	NULL	Last modifier
updated_at	DATETIME2	NULL	Last update
is_deleted	BIT	NOT NULL	Soft delete flag

1.4 Carrier-Specific Features

Partners with `is_carrier = 1` appear in the Shipment module's carrier dropdown. They can be assigned to Fulfillments/Deliveries. Carrier partners have:

- `vehicle_types`: JSON array or comma-separated list of vehicle types (Truck, Van, Bike, etc.)
- Carrier-specific rate cards can be managed through the existing Rate Card structure (Addendum 28)
- Carrier performance can be tracked through the existing Scorecard engine (Addendum 23) with delivery-focused dimensions

1.5 Service Provider Features

Partners with `is_service_provider = 1` can be used for:

- Maintenance service contracts
- Consulting or professional services POs
- Outsourced operations (warehouse labor, transport management)
- `service_categories`: JSON array of service types the provider offers

1.6 API Endpoints

Method	Endpoint	Description
GET	/api/partners	List all partners (filter: type, is_vendor, is_customer, is_carrier, is_service_provider, active, search)
GET	/api/partners/{id}	Get partner detail

Method	Endpoint	Description
POST	/api/partners	Create new partner
PUT	/api/partners/{id}	Update partner
DELETE	/api/partners/{id}	Soft delete partner
GET	/api/partners/{id}/ledger	Partner ledger (payable or receivable based on type)
GET	/api/suppliers	ALIAS -> /api/partners?is_vendor=true (backward compat)
GET	/api/customers	ALIAS -> /api/partners?is_customer=true
GET	/api/carriers	ALIAS -> /api/partners?is_carrier=true
GET	/api/service-providers	ALIAS -> /api/partners?is_service_provider=true

1.7 Backward Compatibility Rules

- All existing PurchaseOrder.supplier_id FKs are renamed to partner_id but continue pointing to the same rows
- The SupplierLedger table is renamed to PartnerLedger; existing entries remain intact
- API route /api/suppliers continues to work as an alias, returning only is_vendor=1 rows
- Existing Supplier Scorecard logic is unchanged; it filters by is_vendor = 1
- No existing data is deleted, moved, or duplicated

Section 2 - Product Pricing Enhancement & Default Supplier

2.1 Default Supplier per Product Variant

Each ProductVariant must have an optional default supplier. This is used by the Back-to-Back PO auto-creation when admin configures 'Use Default Supplier' mode.

Add to existing ProductVariants table:

New columns on inventory.ProductVariants

Column	Type	Nullable	Description
default_supplier_id	BIGINT	NULL	FK to BusinessPartners (is_vendor=1). Default vendor for auto-PO
lead_time_days	INT	NULL	Expected delivery time from default supplier

If no default_supplier_id is set for a variant, the system falls back to the admin-configured supplier selection mode (see Section 3).

2.2 PricingRules Table

inventory.PricingRules

Column	Type	Nullable	Description
pricing_rule_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
variant_id	BIGINT	NOT NULL	FK to ProductVariants
partner_id	BIGINT	NULL	FK to BusinessPartners (NULL = all customers)
price_type	NVARCHAR(20)	NOT NULL	SELLING COST PROMOTIONAL CONTRACT
min_qty	DECIMAL(18,4)	NOT NULL	Minimum quantity for this price (default 1)
max_qty	DECIMAL(18,4)	NULL	Maximum quantity (NULL = unlimited)
unit_price	DECIMAL(18,4)	NOT NULL	Price per unit
currency_code	NVARCHAR(3)	NOT NULL	ISO 4217
effective_from	DATE	NOT NULL	Start date
effective_to	DATE	NULL	End date (NULL = indefinite)
is_active	BIT	NOT NULL	Active flag
created_by	NVARCHAR(100)	NOT NULL	Creator
created_at	DATETIME2	NOT NULL	Created timestamp

2.3 Selling Price Resolution Logic

Priority order when resolving selling price for a Sale Order line:

- Priority 1: CONTRACT price for this specific partner + variant + date range + qty
- Priority 2: PROMOTIONAL price for this variant + date range + qty
- Priority 3: Partner-specific SELLING price for this partner + variant
- Priority 4: Default SELLING price (partner_id IS NULL) for this variant
- Priority 5: Variant.selling_price (fallback from ProductVariants table)

2.4 Purchase Price Resolution (for Back-to-Back PO)

When creating a back-to-back PO, the system must resolve the purchase price. Priority:

- Priority 1: Latest Supplier Rate Card price (Addendum 28) for selected supplier + variant
- Priority 2: Last PO price for this supplier + variant (most recent PurchaseOrderLine)
- Priority 3: Variant.purchase_price (fallback from ProductVariants table)

```
SELECT TOP 1 rate FROM suppliers.SupplierRateCards
WHERE partner_id = @supplierId AND variant_id = @vid
  AND is_active = 1 AND effective_from <= GETDATE()
  AND (effective_to IS NULL OR effective_to >= GETDATE())
ORDER BY effective_from DESC
```

Section 3 - Sale Order Administration & Configuration

All sale order behavior is configurable by the Supply Department Admin. This is NOT the IT Admin or Super Admin. The Supply Department Admin is typically a Deputy Director or Director of Supply who manages procurement and sales operations.

3.1 Admin Role Definition

A new role is created in the auth system:

Property	Value
Role Name	SUPPLY_DEPT_ADMIN
Display Name	Supply Department Administrator
Typical Position	Deputy Director / Director of Supply
Scope	Organization-level (org_id bound)
Parent Permission Group	Supply Chain Management

This role has full control over Sale Order configuration settings. IT Admin and Super Admin can VIEW these settings but cannot CHANGE them unless they also hold SUPPLY_DEPT_ADMIN role.

3.2 Sale Order Configuration Table

demand.SaleOrderConfig

Column	Type	Nullable	Description
config_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator (one config per org)
auto_po_enabled	BIT	NOT NULL	Enable auto Back-to-Back PO creation (default 1)
supplier_selection_mode	NVARCHAR(20)	NOT NULL	DEFAULT_SUPPLIER BEST_MATCH MANUAL
auto_po_approval_mode	NVARCHAR(20)	NOT NULL	REQUIRE_WORKFLOW AUTO_SEND DRAFT_ONLY
drop_ship_enabled	BIT	NOT NULL	Enable Drop Ship fulfillment mode (default 0)
self_pickup_enabled	BIT	NOT NULL	Enable customer self-pickup option (default 1)
default_fulfillment_mode	NVARCHAR(15)	NOT NULL	IN_STOCK BACK_TO_BACK DROP_SHIP (default for new SO lines)
reservation_ttl_hours	INT	NOT NULL	Stock reservation expiry in hours (default 72)
partial_fulfillment_allowed	BIT	NOT NULL	Allow partial picks and deliveries (default 1)
email_intimation_enabled	BIT	NOT NULL	Send email on SO confirmation (default 1)
intimation_department_id	BIGINT	NULL	FK to Departments - receives SO intimation emails
intimation_cc_emails	NVARCHAR(500)	NULL	Additional CC email addresses (comma-separated)
shipment_required_default	BIT	NOT NULL	Default: require shipment for SOs (default 1)
updated_by	NVARCHAR(100)	NOT NULL	Last modifier (must be SUPPLY_DEPT_ADMIN)
updated_at	DATETIME2	NOT NULL	Last update

3.3 Supplier Selection Mode (Detailed)

When `auto_po_enabled = 1` and stock is insufficient, the system must select a supplier. Three modes:

Mode A: DEFAULT_SUPPLIER

- Use the `default_supplier_id` set on the ProductVariant
- If no default supplier is set for a variant, fall back to MANUAL mode for that line
- Simplest mode - admin assigns preferred suppliers per product in advance

Mode B: BEST_MATCH (Recommended)

System uses the existing Supplier Scorecard (Addendum 23) and Rate Card (Addendum 28) to auto-select the best vendor:

- Step 1: Find all active vendors who supply this variant (from `SupplierRateCards` WHERE `variant_id = @vid` AND `is_active = 1`)
- Step 2: For each vendor, retrieve Scorecard grade (A=5, B=4, C=3, D=2, F=1)
- Step 3: For each vendor, retrieve latest rate card price for the required quantity
- Step 4: Calculate composite score = $(\text{Scorecard_Score} * 0.6) + (\text{Price_Rank} * 0.4)$
- Step 5: Select vendor with highest composite score
- Step 6: If tie, prefer vendor with shortest `lead_time_days`

```
-- Best Match Query
WITH VendorCandidates AS (
  SELECT rc.partner_id, rc.rate AS unit_price,
         CASE sc.grade WHEN 'A' THEN 5 WHEN 'B' THEN 4 WHEN 'C' THEN 3 WHEN 'D' THEN 2 ELSE 1
         END AS score,
         RANK() OVER (ORDER BY rc.rate ASC) AS price_rank
  FROM suppliers.SupplierRateCards rc
  LEFT JOIN suppliers.SupplierScorecards sc ON sc.partner_id = rc.partner_id
  WHERE rc.variant_id = @vid AND rc.is_active = 1
)
SELECT TOP 1 partner_id FROM VendorCandidates
ORDER BY (score * 0.6) + ((1.0 / price_rank) * 0.4) DESC
```

Mode C: MANUAL

- Auto-PO is NOT created automatically
- System creates a notification/task for the supply team to manually create the PO
- Used when admin wants human review before every procurement decision

3.4 Auto-PO Approval Mode

Controls what happens after a back-to-back PO is auto-created:

Mode	Behavior	PO Status	Description
REQUIRE_WORKFLOW	PO goes through normal approval workflow	PENDING_APPROVAL	Full workflow approval required before PO is sent to vendor. Supply team may modify qty/price before approval.

Mode	Behavior	PO Status	Description
AUTO_SEND	PO skips approval and is immediately sent to vendor	APPROVED/OPEN	For trusted suppliers and standard items. Fastest but no human review.
DRAFT_ONLY	PO is created as DRAFT only. No workflow triggered.	DRAFT	Supply team must manually review, potentially update qty/price, then submit for approval or send directly. Most conservative.

IMPORTANT: In **DRAFT_ONLY** and **REQUIRE_WORKFLOW** modes, the PO is **NOT** sent to the vendor until manual action is taken. The supply team user can:

- Update the PO quantity (e.g., order more than the SO needs to get bulk pricing)
- Update the supplier (override the auto-selected supplier)
- Update the unit price (negotiate a different rate)
- Add additional items to the PO (combine multiple SO needs into one PO)
- Then submit for approval (**REQUIRE_WORKFLOW**) or manually approve and send (**DRAFT_ONLY**)

3.5 Configuration API Endpoints

Method	Endpoint	Auth	Description
GET	/api/sale-order-config	SUPPLY_DEPT_ADMIN IT_ADMIN (read-only)	Get current config
PUT	/api/sale-order-config	SUPPLY_DEPT_ADMIN only	Update config
GET	/api/sale-order-config/audit	SUPPLY_DEPT_ADMIN	View config change history

Every config change is audit-trailed with who changed what and when.

Section 4 - Sale Orders (3-Scenario Availability Check)

4.1 Sale Order Header

demand.SaleOrders

Column	Type	Nulla ble	Description
sale_order_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
so_number	NVARCHAR(30)	NOT NULL	Auto-generated (SO-YYYYMMDD-NNNN)
partner_id	BIGINT	NOT NULL	FK to BusinessPartners (is_customer=1)
order_date	DATE	NOT NULL	Order placement date
expected_delivery_date	DATE	NULL	Requested delivery date
currency_code	NVARCHAR(3)	NOT NULL	ISO 4217
subtotal	DECIMAL(18,4)	NOT NULL	Sum of line totals before tax
tax_amount	DECIMAL(18,4)	NOT NULL	Total tax
discount_amount	DECIMAL(18,4)	NOT NULL	Header-level discount
grand_total	DECIMAL(18,4)	NOT NULL	subtotal + tax - discount
status	NVARCHAR(20)	NOT NULL	DRAFT CONFIRMED PARTIALLY_FULFILLED FULFILLED INVOICE D CLOSED CANCELLED
requires_shipment	BIT	NOT NULL	1 = ship to customer, 0 = customer self-pickup (from config default, overridable)
delivery_mode	NVARCHAR(15)	NOT NULL	SHIP SELF_PICKUP
shipping_address_id	BIGINT	NULL	FK to logistics.addresses (NULL if self-pickup). Uses existing structured address table.
intimation_department_id	BIGINT	NULL	Department to notify (from config, overridable)
notes	NVARCHAR(MAX)	NULL	Internal notes
trace_id	UNIQUEIDENTIFIER	NOT NULL	Timeline correlation GUID
created_by	NVARCHAR(100)	NOT NULL	Creator
created_at	DATETIME2	NOT NULL	Created timestamp
updated_by	NVARCHAR(100)	NULL	Last modifier
updated_at	DATETIME2	NULL	Last update
is_deleted	BIT	NOT NULL	Soft delete

4.2 Sale Order Lines

demand.SaleOrderLines

Column	Type	Nullable	Description
so_line_id	BIGINT IDENTITY	NOT NULL	PK
sale_order_id	BIGINT	NOT NULL	FK to SaleOrders
variant_id	BIGINT	NOT NULL	FK to ProductVariants
quantity	DECIMAL(18,4)	NOT NULL	Ordered quantity
unit_price	DECIMAL(18,4)	NOT NULL	Resolved selling price
discount_percent	DECIMAL(5,2)	NOT NULL	Line discount %
tax_percent	DECIMAL(5,2)	NOT NULL	Line tax %
line_total	DECIMAL(18,4)	NOT NULL	qty * price * (1-disc%) * (1+tax%)
fulfilled_qty	DECIMAL(18,4)	NOT NULL	Quantity fulfilled so far
invoiced_qty	DECIMAL(18,4)	NOT NULL	Quantity invoiced so far
fulfillment_mode	NVARCHAR(15)	NOT NULL	IN_STOCK BACK_TO_BACK DROP_SHIP SPLIT
available_qty_at_confirmation	DECIMAL(18,4)	NULL	Stock available at confirmation time (for email)
deficit_qty	DECIMAL(18,4)	NULL	Qty that needs procurement (for email)
linked_po_id	BIGINT	NULL	FK to PurchaseOrders (for B2B/DROP_SHIP)
selected_supplier_id	BIGINT	NULL	FK to BusinessPartners - auto/manual selected supplier for B2B
margin	DECIMAL(18,4)	NULL	Selling price - purchase price (informational)
margin_percent	DECIMAL(5,2)	NULL	Margin as %
status	NVARCHAR(20)	NOT NULL	OPEN RESERVED PARTIALLY_FULFILLED FULFILLED INVOICE D CANCELLED
notes	NVARCHAR(500)	NULL	Line notes

4.3 Three-Scenario Availability Check

When a Sale Order is **CONFIRMED**, the system performs an availability check on each line. The fulfillment_mode may be set manually by the user or auto-determined by the system based on stock:

Scenario 1: Fully In-Stock (fulfillment_mode = IN_STOCK)

- Check: available_qty >= ordered_qty
- Action: Reserve stock immediately
- Create StockReservation record linking so_line_id
- Set line status = RESERVED
- Email: 'Product fully available in stock. Reservation created for [qty] units.'

Scenario 2: Partially Available (fulfillment_mode = SPLIT)

When 50% (or any portion) is in stock and the remainder needs procurement:

- Reserve whatever is available in stock immediately
- For the deficit: create a back-to-back PO (per admin config)

- available_qty_at_confirm = stock that was available
- deficit_qty = ordered_qty - available_qty
- Email: 'Product partially available. [X] units reserved from stock. [Y] units require procurement. Purchase Order PO-xxxx created in DRAFT.'
- Two fulfillments will be needed: one from stock, one after GRN receipt

Scenario 3: No Stock / Back-to-Back (fulfillment_mode = BACK_TO_BACK)

- Check: available_qty = 0 or insufficient
- Auto-create Purchase Order for full quantity
- PO status depends on admin config (DRAFT_ONLY / REQUIRE_WORKFLOW / AUTO_SEND)
- When GRN is received, auto-reserve for this SO line
- Email: 'No stock available. Purchase Order PO-xxxx created. Awaiting procurement.'

Scenario 4: Drop Ship (fulfillment_mode = DROP_SHIP)

- Only available if drop_ship_enabled = 1 in config
- Create PO with shipping_address = customer address
- No warehouse stock impact
- Email: 'Drop Ship order. Supplier will ship directly to customer at [address].'

4.4 Stock Reservation (via existing IStockReservationService)

IMPORTANT: The application already has a source-agnostic IStockReservationService in SMS.Shared (G4 decision). Sales reservation MUST use this existing service with ReservationSourceType.SALES_ORDER — do NOT create a separate table or service.

Existing table: inventory.stock_reservations

inventory.stock_reservations (EXISTING — key columns)

Column	Type	Nullable	Description
reservation_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator (HasQueryFilter)
variant_id	BIGINT	NOT NULL	FK to ProductVariants
warehouse_id	BIGINT	NOT NULL	FK to Warehouses
source_type	NVARCHAR(20)	NOT NULL	ReservationSourceType enum: MIR DELIVERY SALES_ORDER
source_id	BIGINT	NOT NULL	FK to source document (SaleOrderLine.so_line_id for SALES_ORDER)
reserved_qty	DECIMAL(18,4)	NOT NULL	Quantity reserved (multi-row FEFO allocation)
status	NVARCHAR(15)	NOT NULL	ACTIVE RELEASED CONSUMED EXPIRED
row_version	ROWVERSION	NOT NULL	Concurrency token — prevents double-reserve
reserved_at	DATETIME2	NOT NULL	Reservation timestamp
expires_at	DATETIME2	NULL	Auto-expiry (from config TTL)

Service contract (already implemented in SMS.Shared):

```
public interface IStockReservationService
{
    Task<ReservationResult> Reserve(ReservationRequest request);
    Task Release(long reservationId);
    Task Consume(long reservationId);
}
```

For Sale Order confirmation, call:

```
await _reservationService.Reserve(new ReservationRequest {
    OrgId = currentUser.OrgId,
    VariantId = soLine.variant_id,
    WarehouseId = warehouseId,
    SourceType = ReservationSourceType.SALES_ORDER,
    SourceId = soLine.so_line_id,
    Quantity = availableQty,
    ExpiresAt = DateTime.UtcNow.AddHours(config.reservation_ttl_hours)
});
```

Multi-row FEFO allocation: If the required quantity spans multiple stock batches (different expiry dates), the service creates multiple reservation rows, one per batch, allocating first-expiring-first-out. The counter never goes negative.

4.5 Sale Order API Endpoints

Method	Endpoint	Description
GET	/api/sale-orders	List SOs (filter: status, customer, date range)
GET	/api/sale-orders/{id}	Get SO detail with lines
POST	/api/sale-orders	Create new SO (DRAFT)
PUT	/api/sale-orders/{id}	Update SO (only while DRAFT)
POST	/api/sale-orders/{id}/confirm	Confirm SO - triggers availability check + email
POST	/api/sale-orders/{id}/cancel	Cancel SO - releases reservations
GET	/api/sale-orders/{id}/timeline	Get SO timeline events
GET	/api/sale-orders/{id}/availability	Preview availability check without confirming

Section 5 - Sale Order Email Intimation System

5.1 When Emails Are Sent

Emails are triggered via Hangfire BackgroundJob.Enqueue (non-blocking) at these events:

Event	Recipients	Content Summary
SO Confirmed	Intimation Department + CC list	SO details, per-line availability status, fulfillment mode, any auto-POs created
Back-to-Back PO Created	Intimation Department	PO details, linked SO, supplier selected, price, status (DRAFT/PENDING)
Drop Ship Detected	Intimation Department	Drop ship notification, supplier, customer delivery address
Stock Reserved	SO Creator	Confirmation of stock reservation with expiry time
PO Approved (manual)	SO Creator + Intimation Dept	PO approved and sent to vendor, expected delivery
GRN Received (linked to SO)	SO Creator	Stock received, auto-reserved for SO, ready for fulfillment
Reservation Expiring	SO Creator	Warning: reservation expires in 24 hours

5.2 Email Template - SO Confirmation

Subject: [SMS] Sale Order {SO-Number} Confirmed - Availability Summary

The email body contains:

- Header: SO Number, Customer Name, Order Date, Expected Delivery Date
- Line-by-line breakdown table:

Product/Variant	Ordered Qty	Available	Deficit	Fulfillment Mode	Action Taken
Dell i7/16/512	100	60	40	SPLIT	60 reserved from stock. PO-2026-0045 created (DRAFT) for 40 units from TechSupply Co.
Mouse Wireless	200	200	0	IN_STOCK	200 units reserved from warehouse WH-01.
Monitor 27in	50	0	50	DROP_SHIP	PO-2026-0046 created (DRAFT). Supplier ships directly to customer.

- Footer: Action items (e.g., 'PO-2026-0045 requires approval before sending to vendor')
- If self-pickup: 'Customer will collect. No shipment required. Quantities will be held at [warehouse].'

5.3 Email Intimation Table

demand.SaleOrderIntimations

Column	Type	Null able	Description
intimation_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
sale_order_id	BIGINT	NOT NULL	FK to SaleOrders
event_type	NVARCHAR(30)	NOT NULL	SO_CONFIRMED PO_CREATED DROP_SHIP RESERVED PO_APPROVED GRN_RECEIVED EXPIRING
recipients	NVARCHAR(500)	NOT NULL	Email addresses sent to
subject	NVARCHAR(200)	NOT NULL	Email subject

Column	Type	Nullable	Description
body_html	NVARCHAR(MAX)	NOT NULL	Full email body (HTML)
sent_at	DATETIME2	NOT NULL	When sent
status	NVARCHAR(15)	NOT NULL	SENT FAILED QUEUED
error_message	NVARCHAR(500)	NULL	If FAILED, the error
hangfire_job_id	NVARCHAR(50)	NULL	Hangfire job reference

All emails are logged for audit. Failed emails are retried up to 3 times by Hangfire.

Section 6 - Back-to-Back PO (Draft + Approval Control)

6.1 Auto-PO Creation Flow

When availability check identifies a deficit:

```
SO Line (deficit detected)
|
+--> Supplier Selection (per admin config mode)
|   DEFAULT_SUPPLIER -> use variant.default_supplier_id
|   BEST_MATCH -> scorecard + rate card composite score
|   MANUAL -> create task for supply team, no auto-PO
|
+--> Price Resolution (Rate Card -> Last PO -> Variant.purchase_price)
|
+--> Create PO (status per admin config)
|   DRAFT_ONLY -> PO status = DRAFT (user reviews, may edit qty/price)
|   REQUIRE_WORKFLOW -> PO status = PENDING_APPROVAL (enters workflow)
|   AUTO_SEND -> PO status = APPROVED (sent to vendor immediately)
|
+--> Send Email Intimation to department
```

6.2 PO Modifications Before Approval

In **DRAFT_ONLY** and **REQUIRE_WORKFLOW** modes, the supply team can modify the PO before it is sent to the vendor:

- Update quantity: User may want to order more than the SO deficit (e.g., order 100 when SO needs 40 to get bulk pricing)
- Update supplier: Override the auto-selected supplier with a different vendor
- Update unit price: Negotiate a different rate than what the system resolved
- Add items: Combine multiple SO deficit needs into a single PO
- Split: Break one PO into multiple POs for different suppliers

The system tracks the original auto-generated values vs. the modified values for audit purposes.

6.3 New PO Fields (added to existing PurchaseOrders)

New columns on procurement.PurchaseOrders

Column	Type	Nullable	Description
source	NVARCHAR(20)	NOT NULL	MANUAL BACK_TO_BACK DROP_SHIP MIR (default MANUAL)
linked_so_line_id	BIGINT	NULL	FK to SaleOrderLines (for B2B and Drop Ship)
linked_so_id	BIGINT	NULL	FK to SaleOrders (parent reference)
customer_shipping_address_id	BIGINT	NULL	FK to logistics.addresses — for Drop Ship, customer delivery address
auto_generated_qty	DECIMAL(18,4)	NULL	Original system-generated qty (before user edits)

Column	Type	Nullable	Description
auto_generated_price	DECIMAL(18,4)	NULL	Original system-resolved price (before user edits)
auto_selected_supplier_id	BIGINT	NULL	Original system-selected supplier (before user override)

6.3.1 trace_id Propagation (SO → PO)

When a back-to-back or drop-ship PO is auto-created from a Sale Order, the SO's trace_id is propagated to the PO. This follows the SAME pattern already implemented for Quotation→PO and PR→PO linkage:

```
// During auto-PO creation from SO confirmation:
newPO.trace_id = saleOrder.trace_id;    // SAME correlation GUID
newPO.source = 'BACK_TO_BACK';          // or 'DROP_SHIP'
newPO.linked_so_id = saleOrder.sale_order_id;
newPO.linked_so_line_id = soLine.so_line_id;

// Hangfire non-blocking timeline write:
BackgroundJob.Enqueue(() => _timelineService.AppendEvent(
    saleOrder.trace_id,
    new TimelineEvent {
        event_type = 'PO_CREATED_FROM_SO',
        source_type = 'SALE_ORDER',
        source_id = saleOrder.sale_order_id,
        source_number = saleOrder.so_number,
        target_type = 'PURCHASE_ORDER',
        target_id = newPO.purchase_order_id,
        target_number = newPO.po_number,
        timestamp = DateTime.UtcNow,
        actor = currentUser,
        details = $"{soLine.quantity} units of {variant.sku} → {supplier.company_name}"
    }));
```

This means a single trace_id can now correlate the full chain: Quotation → PO → GRN → Invoice (procurement side) AND Sale Order → PO → GRN → Reservation → Fulfillment → Sales Invoice (sales side). The DocumentTimelines table stores all events in one JSON array per trace_id.

6.4 GRN Auto-Reservation Link

When a GRN is received against a back-to-back PO that is linked to a SO line:

- Step 1: GRN received - stock enters warehouse
- Step 2: System checks if PO has linked_so_line_id
- Step 3: If yes, auto-create StockReservation for the received qty against the SO line
- Step 4: Update SO line status from OPEN -> RESERVED
- Step 5: Send email to SO creator: 'Stock received for your order. Ready for fulfillment.'
- Step 6: If partial GRN, only partial reservation is created; SO line remains OPEN until fully received

Section 7 - Fulfillment via Existing Delivery Orders

CRITICAL: The application already has a fully implemented four-layer logistics schema with **delivery_orders**, pick lists, packages, consignments, goods issue, and carrier integration (1531 tests). Sales fulfillment **MUST** use this existing infrastructure — do **NOT** create separate warehouse.Fulfillments tables.

7.1 Reuse Strategy: delivery_orders with from_source_type = SALE_ORDER

The existing logistics.delivery_orders table supports creation from multiple source types (PO, SRO, MIV, Transfer). Add SALE_ORDER as a new from_source_type to enable sales fulfillment through the same infrastructure:

New columns to add to existing logistics.delivery_orders:

New columns on logistics.delivery_orders (additions only)

Column	Type	Nullable	Description
from_source_type	NVARCHAR(20)	NOT NULL	EXISTING column — add SALE_ORDER to allowed values
sale_order_id	BIGINT	NULL	FK to demand.SaleOrders (when from_source_type = SALE_ORDER)
delivery_mode	NVARCHAR(15)	NULL	SHIP SELF_PICKUP (only for SALE_ORDER source)
pickup_person_name	NVARCHAR(100)	NULL	Self-pickup: who collected
pickup_person_id_type	NVARCHAR(30)	NULL	Self-pickup: ID type (CNIC/License/Passport)
pickup_person_id_number	NVARCHAR(50)	NULL	Self-pickup: ID number
pickup_authorization	NVARCHAR(200)	NULL	Self-pickup: authorization reference from customer

New column on existing logistics.delivery_order_lines:

New column on logistics.delivery_order_lines

Column	Type	Nullable	Description
so_line_id	BIGINT	NULL	FK to demand.SaleOrderLines (when parent DO is SALE_ORDER source)

7.2 Existing Delivery State Machine (15 Statuses)

Sales fulfillment uses the **SAME** state machine already implemented:

DRAFT → RELEASED → PICKING → PICKED → PACKED → STAGED
 → PENDING_APPROVAL → GOODS_ISSUED → IN_TRANSIT
 → AT_HUB → OUT_FOR_DELIVERY → DELIVERY_ATTEMPTED
 → DELIVERED → COMPLETED → CANCELLED

For sales shipments, the typical path is:

DRAFT → RELEASED → PICKING → PICKED → PACKED → GOODS_ISSUED → IN_TRANSIT → DELIVERED

For self-pickup, the path is:

DRAFT → RELEASED → PICKING → PICKED → PACKED → GOODS_ISSUED → DELIVERED

(Self-pickup skips IN_TRANSIT — goods issue posts the SALES_HANDBOVER movement, then status advances to DELIVERED when customer collects)

7.3 Creating Delivery Orders from Sale Orders

Use the existing from-source creation pattern (same as PO/SRO/MIV/Transfer):

```
// DeliveryOrderService.CreateFromSaleOrder(saleOrderId, lines)
var deliveryOrder = new DeliveryOrder {
    OrgId = currentUser.OrgId,
    DeliveryNumber = await _numberService.NextNumber('DO', orgId),
    FromSourceType = DeliverySourceType.SALE_ORDER,
    SaleOrderId = saleOrder.SaleOrderId,
    WarehouseId = warehouseId,
    DeliveryMode = saleOrder.DeliveryMode, // SHIP or SELF_PICKUP
    ShipToAddressId = saleOrder.ShippingAddressId, // FK to logistics.addresses
    Status = DeliveryStatus.DRAFT,
    TraceId = saleOrder.TraceId // propagate timeline correlation
};
```

7.4 Leveraging Existing Pick/Pack/Ship Infrastructure

All existing logistics infrastructure is reused as-is:

- Pick Lists: FEFO/FIFO allocation via existing pick_list_lines with zone+bin walk order (T-22)
- Pack Station: Existing package API with HU barcodes, nesting (carton → pallet), void-not-delete (T-26)
- Goods Issue: Existing PostsGoodsIssue determines movement type from delivery source — add SALE_ORDER case posting SALES_SHIP or SALES_HANDBOVER movement (T-27)
- Carrier Booking: Existing ICourierProvider.BookAsync via consignments — same booking orchestration via Hangfire (T-37)
- Tracking: Existing webhook + poll tracking via TrackingEventRecorder (T-42/T-43)
- Labels: Existing consignment_labels with ICourierProvider.LabelAsync (T-39)
- Proof of Delivery: Existing delivery_proofs with bytes-stored-not-URLs pattern (T-61)

7.5 Goods Issue for Sales Source Type

Extend the existing PostsGoodsIssue switch to handle SALE_ORDER source:

```
// In GoodsIssueService – existing switch on from_source_type:
case DeliverySourceType.SALE_ORDER:
    movementType = deliveryOrder.DeliveryMode == 'SELF_PICKUP'
        ? MovementType.SALES_HANDBOVER
        : MovementType.SALES_SHIP;
    // Consume reservation via IStockReservationService
    await _reservationService.Consume(reservationId);
    // Update SO line fulfilled_qty
    soLine.FulfilledQty += issuedQty;
    break;
```

Goods issue is idempotent (existing pattern) — double-post returns same result.

7.6 Partial Fulfillment (Multiple Delivery Orders)

A single Sale Order can generate MULTIPLE Delivery Orders. The existing partial delivery pattern supports this:

SO Line	Variant	Ordered	Delivery Order 1	Delivery Order 2	Delivery Order 3
SOL-001	Dell i7/16/512	100	60 (from stock, shipped)	30 (from GRN, shipped)	10 (from GRN, self-pickup)
SOL-002	Mouse Wireless	200	200 (from stock, shipped)	-	-

The SO status transitions:

- CONFIRMED → PARTIALLY_FULFILLED (when first delivery order reaches DELIVERED/COMPLETED)
- PARTIALLY_FULFILLED → FULFILLED (when all lines have fulfilled_qty = ordered_qty)
- Each delivery order is independent: different delivery modes, different carriers, different dates

7.7 Shipping Address (logistics.addresses)

Use the existing logistics.addresses table for structured shipping addresses instead of flat NVARCHAR(500):

The SaleOrders table should reference shipping_address_id (FK to logistics.addresses) instead of storing a flat string. The addresses table already stores line1, line2, city, state, country, postal_code, geo coordinates, and contact info.

7.8 Sales Fulfillment API Endpoints

Sales fulfillment uses the EXISTING delivery order API endpoints. New convenience endpoints:

Method	Endpoint	Description
POST	/api/sale-orders/{id}/create-delivery	Create delivery order from SO (specify lines + qtys) — calls existing DeliveryOrderService.CreateFromSource
GET	/api/sale-orders/{id}/deliveries	List delivery orders for this SO (filter on sale_order_id)
POST	/api/delivery-orders/{id}/pickup	Record self-pickup details (person name, ID, authorization) then advance to DELIVERED

All other delivery operations (pick, pack, goods issue, ship, track, deliver) use existing /api/delivery-orders/{id}/* endpoints.

7.9 Permissions

Sales fulfillment uses existing granular permissions:

Permission	Required For
DELIVERY_VIEW	View delivery orders created from SOs
DELIVERY_EDIT	Edit delivery order details
DELIVERY_RELEASE	Release delivery order for picking
PICKING	Start and confirm pick lists
PACKING	Pack items into shipment packages
DISPATCH / GOODS_ISSUE	Post goods issue and dispatch
SHIPMENT_BOOK	Book consignment with carrier
POD_CAPTURE	Record proof of delivery

Section 8 - Self-Pickup Flow (via Delivery Order + Goods Issue)

When `delivery_mode = SELF_PICKUP` on the Sale Order, the customer arranges their own collection. This uses the SAME `delivery_orders` infrastructure with the existing goods issue pattern (T-27).

8.1 Configuration Privilege

The Supply Department Admin controls whether self-pickup is available:

- `SaleOrderConfig.self_pickup_enabled = 1`: Users can set `delivery_mode = SELF_PICKUP` on any SO
- `SaleOrderConfig.self_pickup_enabled = 0`: All SOs must use SHIP mode
- `SaleOrderConfig.shipment_required_default = 0`: New SOs default to `SELF_PICKUP` (user can override)

8.2 Self-Pickup Process (Using Existing Delivery State Machine)

Step	Delivery Status	Action	Stock Impact
1	—	SO created with <code>delivery_mode = SELF_PICKUP</code> , no shipping address	None
2	—	SO confirmed — <code>IStockReservationService.Reserve(SALES_ORDER)</code>	<code>reserved_qty</code> increased
3	DRAFT	Delivery Order created (<code>from_source_type = SALE_ORDER</code> , <code>delivery_mode = SELF_PICKUP</code>)	None
4	RELEASED	DO released for warehouse execution	None
5	PICKING	Pick list generated (FEFO, zone+bin walk order) via existing pick service	Pick allocation recorded
6	PICKED	Pick confirmed — short-pick reasons tracked if partial	None
7	PACKED	Items packed into <code>shipment_packages</code> (optional for self-pickup)	None
8	GOODS_ISSUED	<code>PostsGoodsIssue</code> → <code>SALES_HANDOVER</code> movement. Reservation consumed.	<code>on_hand</code> decreased, reservation consumed
9	—	Customer notified via email: ready for collection at [warehouse]	None
10	DELIVERED	Customer arrives → record <code>pickup_person_name</code> , ID, authorization → advance to DELIVERED	SO line <code>fulfilled_qty</code> updated

Key difference from shipment flow: Self-pickup skips IN_TRANSIT and carrier booking. Goods Issue posts SALES_HANDOVER instead of SALES_SHIP. No consignment is created.

8.3 Quantity Movement Documentation

The existing goods issue pattern handles movement documentation:

- Goods Issue posts the stock movement (`SALES_HANDOVER`) — this is the audit trail
- A Gate Pass / Packing List is auto-generated via QuestPDF (reuse existing packing list PDF from T-28)
- The gate pass contains: `delivery_order_number`, item list, quantities, pickup person details, authorization, timestamp
- The gate pass is stored as a document attachment on the Delivery Order record
- Warehouse security uses the gate pass for outbound verification

```
// Goods issue for self-pickup (in GoodsIssueService):
case DeliverySourceType.SALE_ORDER when delivery.DeliveryMode == 'SELF_PICKUP':
```

```
// Post SALES_HANDOVER movement
await PostMovement(MovementType.SALES_HANDOVER, delivery);
// Consume reservation
await _reservationService.Consume(reservation.ReservationId);
// Update SO line
soLine.FulfilledQty += issuedQty;
break;
```

Section 9 - Sales Invoicing & Customer Payments

9.1 Sales Invoice Header

finance.SalesInvoices

Column	Type	Nullable	Description
invoice_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
invoice_number	NVARCHAR(30)	NOT NULL	Auto-generated (SINV-YYYYMMDD-NNNN)
sale_order_id	BIGINT	NOT NULL	FK to SaleOrders
partner_id	BIGINT	NOT NULL	FK to BusinessPartners (customer)
invoice_date	DATE	NOT NULL	Invoice issue date
due_date	DATE	NOT NULL	Payment due date
subtotal	DECIMAL(18,4)	NOT NULL	Sum of line totals
tax_amount	DECIMAL(18,4)	NOT NULL	Total tax
discount_amount	DECIMAL(18,4)	NOT NULL	Total discount
grand_total	DECIMAL(18,4)	NOT NULL	Net payable
amount_paid	DECIMAL(18,4)	NOT NULL	Total received
balance_due	DECIMAL(18,4)	NOT NULL	grand_total - amount_paid
status	NVARCHAR(20)	NOT NULL	DRAFT ISSUED PARTIALLY_PAID PAID OVERDUE CANCELLED CREDIT_NOTE
currency_code	NVARCHAR(3)	NOT NULL	ISO 4217
trace_id	UNIQUEIDENTIFIER	NOT NULL	Timeline correlation
notes	NVARCHAR(MAX)	NULL	Invoice notes
created_by	NVARCHAR(100)	NOT NULL	Creator
created_at	DATETIME2	NOT NULL	Created timestamp
is_deleted	BIT	NOT NULL	Soft delete

9.2 Sales Invoice Lines

finance.SalesInvoiceLines

Column	Type	Nullable	Description
invoice_line_id	BIGINT IDENTITY	NOT NULL	PK
invoice_id	BIGINT	NOT NULL	FK to SalesInvoices
so_line_id	BIGINT	NOT NULL	FK to SaleOrderLines
variant_id	BIGINT	NOT NULL	FK to ProductVariants
description	NVARCHAR(300)	NOT NULL	Line description
quantity	DECIMAL(18,4)	NOT NULL	Invoiced quantity
unit_price	DECIMAL(18,4)	NOT NULL	Unit price

Column	Type	Nullable	Description
discount_percent	DECIMAL(5,2)	NOT NULL	Line discount
tax_percent	DECIMAL(5,2)	NOT NULL	Line tax
line_total	DECIMAL(18,4)	NOT NULL	Computed line total

9.3 Customer Payment Recording

finance.CustomerPayments

Column	Type	Nullable	Description
payment_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
partner_id	BIGINT	NOT NULL	FK to BusinessPartners
payment_number	NVARCHAR(30)	NOT NULL	Auto-generated (CPAY-YYYYMMDD-NNNN)
payment_date	DATE	NOT NULL	Date of receipt
amount	DECIMAL(18,4)	NOT NULL	Total amount received
payment_method	NVARCHAR(30)	NOT NULL	CASH CHEQUE BANK_TRANSFER CARD ONLINE
cheque_number	NVARCHAR(50)	NULL	Cheque # if cheque
bank_reference	NVARCHAR(100)	NULL	Bank transaction ref
currency_code	NVARCHAR(3)	NOT NULL	ISO 4217
notes	NVARCHAR(500)	NULL	Payment notes
status	NVARCHAR(15)	NOT NULL	RECEIVED BOUNCED REVERSED
created_by	NVARCHAR(100)	NOT NULL	Creator
created_at	DATETIME2	NOT NULL	Created timestamp

9.4 Payment Allocation

finance.PaymentAllocations

Column	Type	Nullable	Description
allocation_id	BIGINT IDENTITY	NOT NULL	PK
payment_id	BIGINT	NOT NULL	FK to CustomerPayments
invoice_id	BIGINT	NOT NULL	FK to SalesInvoices
allocated_amount	DECIMAL(18,4)	NOT NULL	Amount applied to this invoice
allocated_at	DATETIME2	NOT NULL	Allocation timestamp
allocated_by	NVARCHAR(100)	NOT NULL	User who allocated

One payment can be allocated across multiple invoices. Auto-allocates FIFO (oldest unpaid first) with manual override.

9.5 Invoicing Business Rules

- Invoices can only be created against DELIVERED/PICKED_UP fulfillment lines
- Invoiced quantity cannot exceed delivered/picked-up quantity
- Multiple invoices per SO supported (partial invoicing)
- When invoice ISSUED: CustomerLedger DEBIT entry in same transaction
- When payment received: CustomerLedger CREDIT entry in same transaction
- Invoice auto-updates: PARTIALLY_PAID when amount_paid > 0, PAID when balance_due = 0
- OVERDUE set by Hangfire job scanning due_date < today AND balance_due > 0

Section 10 - Customer Ledger (Receivables)

Mirrors the existing SupplierLedger (payables) but tracks receivables. Same append-only pattern with running balance, written in same transaction as business action.

finance.CustomerLedger

Column	Type	Nullable	Description
entry_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
partner_id	BIGINT	NOT NULL	FK to BusinessPartners (customer)
entry_date	DATETIME2	NOT NULL	Transaction date
entry_type	NVARCHAR(30)	NOT NULL	INVOICE PAYMENT CREDIT_NOTE DEBIT_NOTE ADVANCE REFUND OPENING_BAL
reference_type	NVARCHAR(30)	NOT NULL	SALES_INVOICE CUSTOMER_PAYMENT CREDIT_NOTE etc.
reference_id	BIGINT	NOT NULL	FK to source document
reference_number	NVARCHAR(50)	NOT NULL	Source document number
debit_amount	DECIMAL(18,4)	NOT NULL	Increases receivable (invoice issued)
credit_amount	DECIMAL(18,4)	NOT NULL	Decreases receivable (payment received)
running_balance	DECIMAL(18,4)	NOT NULL	Cumulative balance (debit - credit)
currency_code	NVARCHAR(30)	NOT NULL	ISO 4217
narration	NVARCHAR(500)	NOT NULL	Human-readable description
created_by	NVARCHAR(100)	NOT NULL	Creator
created_at	DATETIME2	NOT NULL	Created timestamp

10.1 Ledger Entry Direction (Receivables)

Event	Debit	Credit	Effect on Balance
Sales Invoice Issued	grand_total	0	Balance increases (customer owes more)
Customer Payment Received	0	amount	Balance decreases (customer paid)
Credit Note Issued	0	credit_amount	Balance decreases (we owe customer)
Debit Note Issued	debit_amount	0	Balance increases (customer owes more)
Customer Advance	0	advance_amount	Balance decreases (prepayment)
Refund Given	refund_amount	0	Balance increases (we refunded, customer may still owe)
Opening Balance	amount	0	Sets initial receivable

Running balance formula: $\text{previous_running_balance} + \text{debit_amount} - \text{credit_amount}$

Section 11 - Product Ledger

The Product Ledger tracks all financial events related to a specific product variant - both purchase (cost) side and sales (revenue) side. This provides a per-product P&L view.

11.1 Product Ledger Table

finance.ProductLedger

Column	Type	Nullabl e	Description
entry_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
variant_id	BIGINT	NOT NULL	FK to ProductVariants
product_id	BIGINT	NOT NULL	FK to Products (denormalized for reporting)
entry_date	DATETIME2	NOT NULL	Transaction date
entry_type	NVARCHAR(30)	NOT NULL	PURCHASE SALE RETURN_IN RETURN_OUT ADJUSTMENT WRITE_OFF
reference_type	NVARCHAR(30)	NOT NULL	GRN SALES_INVOICE RETURN ADJUSTMENT etc.
reference_id	BIGINT	NOT NULL	FK to source document
reference_number	NVARCHAR(50)	NOT NULL	Source document number
partner_id	BIGINT	NULL	FK to BusinessPartners (supplier or customer)
quantity	DECIMAL(18,4)	NOT NULL	Quantity transacted
unit_cost	DECIMAL(18,4)	NOT NULL	Cost/price per unit
total_cost	DECIMAL(18,4)	NOT NULL	quantity * unit_cost
direction	NVARCHAR(5)	NOT NULL	IN (purchase/return-in) OUT (sale/return-out)
running_qty	DECIMAL(18,4)	NOT NULL	Cumulative quantity balance
running_value	DECIMAL(18,4)	NOT NULL	Cumulative inventory value (weighted avg cost)
narration	NVARCHAR(50)	NOT NULL	Human-readable description
created_by	NVARCHAR(10)	NOT NULL	Creator
created_at	DATETIME2	NOT NULL	Created timestamp

11.2 When Product Ledger Entries Are Written

Event	Entry Type	Direction	Trigger
GRN Approved	PURCHASE	IN	Stock received from supplier - cost = PO line unit_price
Sales Invoice Issued	SALE	OUT	Product sold to customer - cost = weighted avg cost, revenue = selling price
Customer Return	RETURN_IN	IN	Product returned by customer - reverses the sale entry
Supplier Return	RETURN_OUT	OUT	Product returned to supplier - reverses the purchase entry
Stock Adjustment	ADJUSTMENT	IN/OUT	Physical count adjustment
Write Off	WRITE_OFF	OUT	Damaged/expired stock written off

11.3 Weighted Average Cost Calculation

The Product Ledger maintains weighted average cost (WAC) for inventory valuation:

On PURCHASE (IN):

```
new_running_qty = prev_running_qty + received_qty
new_running_value = prev_running_value + (received_qty * unit_cost)
new_WAC = new_running_value / new_running_qty
```

On SALE (OUT):

```
COGS_per_unit = current WAC (running_value / running_qty)
new_running_qty = prev_running_qty - sold_qty
new_running_value = prev_running_value - (sold_qty * COGS_per_unit)
```

Product Ledger entries are written in the SAME transaction as the business action (GRN approval, Invoice issuance). This is the same pattern used for Customer Ledger and Supplier Ledger.

11.4 Product Ledger API Endpoints

Method	Endpoint	Description
GET	/api/product-ledger/{variantId}	Ledger entries for a specific variant (paginated)
GET	/api/product-ledger/{variantId}/summary	Summary: total purchased, total sold, current stock value, WAC
GET	/api/reports/product-ledger	All variants ledger report (filter: product, category, date range)
GET	/api/reports/product-profitability	Revenue vs COGS by product (margin report)

Section 12 - Inventory Impact (Sales Side)

Sales-side inventory changes use the EXISTING goods issue and stock reservation infrastructure. The IStockReservationService (G4) already manages reserved_qty on stock_balances.

12.1 Stock Movement Types for Sales

Add these movement types to the existing MovementType enum (used by GoodsIssueService):

Movement Type	Direction	Trigger	Existing Pattern Reference
SALES_SHIP	OUT	Goods Issue (delivery_mode=SHIP)	Same as existing GI for PO/SRO — source type determines movement type (T-27)
SALES_HANDOVER	OUT	Goods Issue (delivery_mode=SELF_PICKUP)	Same goods issue path, different movement type label
SALES_RETURN	IN	Customer Return Receipt	Reverse of SALES_SHIP/HANDOVER
DROP_SHIP_VIRTUAL	N/A	Drop Ship GRN	Virtual receipt+dispatch (no physical warehouse stock affected)

Note: SALES_RESERVE and SALES_UNRESERVE are NOT separate movement types. Reservation is handled by IStockReservationService.Reserve/Release/Consume which directly manages the reserved_qty on inventory.stock_balances. The pick movement is handled by the existing pick list service.

12.2 StockBalances (Already Extended by G4)

The existing inventory.stock_balances table already includes reserved_qty and available_qty columns added by the IStockReservationService implementation (G4 decision):

```
-- Already exists from G4 implementation:
on_hand_qty    -- physical stock in warehouse
reserved_qty   -- stock committed across ALL reservation sources (MIR + DELIVERY + SALES_ORDER)
available_qty   -- on_hand_qty - reserved_qty (computed or maintained)
```

Sales reservation (ReservationSourceType.SALES_ORDER) shares the same reserved_qty counter with MIR and DELIVERY reservations. The IStockReservationService ensures the counter never goes negative and uses FEFO allocation across multiple stock batches.

Section 13 - PO-SO Linkage via trace_id / DocumentTimeline

Before this addendum, Purchase Orders were already linked to Quotations and Purchase Requisitions via the trace_id / DocumentTimelines pattern. Now POs will ALSO be linked to Sale Orders using the exact same mechanism.

13.1 Existing Linkage Pattern (Reference)

The established trace_id pattern works as follows:

- Each originating document (PR, Quotation, SO) generates a trace_id (UNIQUEIDENTIFIER / GUID) at creation
- When a downstream document (PO) is created from the originating document, the same trace_id is propagated
- The workflow.DocumentTimelines table stores a JSON event array per trace_id, recording every document transition
- Timeline writes are NON-BLOCKING via Hangfire BackgroundJob.Enqueue — the business transaction commits first, timeline recording happens asynchronously

13.2 DocumentTimelines Table (Existing)

workflow.DocumentTimelines

Column	Type	Nullable	Description
timeline_id	BIGINT IDENTITY	NOT NULL	PK
org_id	INT	NOT NULL	Tenant discriminator
trace_id	UNIQUEIDENTIFIER	NOT NULL	Correlation GUID (indexed, used for lookups)
events	NVARCHAR(MAX)	NOT NULL	JSON array of timeline events [{event_type, source_type, source_id, ...}]
created_at	DATETIME2	NOT NULL	First event timestamp
updated_at	DATETIME2	NOT NULL	Last event timestamp

13.3 New Timeline Events for SO→PO Chain

The following new event_type values are added to support the SO→PO linkage:

event_type	Trigger	source_type	target_type	Description
SO_CREATED	Sale Order created (DRAFT)	SALE_ORDER	-	Chain origin when SO initiates procurement
SO_CONFIRMED	Sale Order confirmed	SALE_ORDER	-	Availability check completed, reservations/POs created
PO_CREATED_FROM_SO	Auto-PO created from SO deficit	SALE_ORDER	PURCHASE_ORDER	Links SO line to auto-generated PO (B2B or Drop Ship)
GRN_FOR_SO_PO	GRN received for SO-linked PO	PURCHASE_ORDER	GRN	Stock received for a back-to-back PO, auto-reservation follows
SO_STOCK_RESERVED	Stock reserved (from stock or GRN)	SALE_ORDER	STOCK_RESERVATION	Inventory committed to SO line

event_type	Trigger	source_type	target_type	Description
SO_FULFILLED	Fulfillment delivered/picked up	SALE_ORDER	FULFILLMENT	Goods handed over to customer
SO_INVOICED	Sales invoice issued	SALE_ORDER	SALES_INVOICE	Revenue recognized

13.4 Worked Example: Full Chain

Scenario: Customer orders 100 units. 60 available in stock, 40 need procurement (SPLIT mode).

Step 1: Sale Order SO-20260915-0001 created with trace_id = 'a1b2c3d4-...'

```
Event: { event_type: 'SO_CREATED', source_type: 'SALE_ORDER',
        source_id: 1001, source_number: 'SO-20260915-0001',
        timestamp: '2026-09-15T10:00:00Z', actor: 'sales.user1' }
```

Step 2: SO confirmed — 60 reserved from stock, PO created for 40 deficit

```
Event: { event_type: 'SO_CONFIRMED', source_type: 'SALE_ORDER',
        source_id: 1001, source_number: 'SO-20260915-0001',
        details: '60 reserved, 40 deficit → PO created' }
```

```
Event: { event_type: 'SO_STOCK_RESERVED', source_type: 'SALE_ORDER',
        source_id: 1001, target_type: 'STOCK_RESERVATION',
        target_id: 501, details: '60 units reserved from WH-01' }
```

```
Event: { event_type: 'PO_CREATED_FROM_SO', source_type: 'SALE_ORDER',
        source_id: 1001, source_number: 'SO-20260915-0001',
        target_type: 'PURCHASE_ORDER', target_id: 2001,
        target_number: 'PO-20260915-0088',
        details: '40 units of SKU-DELL-i7 → TechSupply Co (DRAFT)' }
```

Step 3: PO approved and sent to vendor

```
Event: { event_type: 'PO_APPROVED', source_type: 'PURCHASE_ORDER',
        source_id: 2001, source_number: 'PO-20260915-0088' }
```

Step 4: GRN received, auto-reservation triggered

```
Event: { event_type: 'GRN_FOR_SO_PO', source_type: 'PURCHASE_ORDER',
        source_id: 2001, target_type: 'GRN', target_id: 3001,
        target_number: 'GRN-20260920-0012',
        details: '40 units received, auto-reserved for SO-20260915-0001' }
```

```
Event: { event_type: 'SO_STOCK_RESERVED', source_type: 'SALE_ORDER',
        source_id: 1001, target_type: 'STOCK_RESERVATION',
        target_id: 502, details: '40 units reserved from GRN receipt' }
```

Step 5: Fulfillments, invoice — all sharing the SAME trace_id

```
Event: { event_type: 'SO_FULFILLED', ... }
```

```
Event: { event_type: 'SO_INVOICED', ... }
```

All events are stored in a **SINGLE JSON array** in `DocumentTimelines.events` for `trace_id = 'a1b2c3d4-...'`. This enables the full end-to-end timeline query: `GET /api/sale-orders/{id}/timeline` returns every linked document in chronological order.

13.5 API: Timeline Query

Method	Endpoint	Description
GET	/api/sale-orders/{id}/timeline	Returns the full DocumentTimeline events array for the SO's trace_id. Includes all linked POs, GRNs, Reservations, Fulfillments, Invoices.
GET	/api/purchase-orders/{id}/timeline	Same timeline from PO perspective — trace_id is shared, so returns SO events too
GET	/api/timeline/{traceId}	Direct timeline lookup by trace_id (generic endpoint)

13.6 Implementation Rules

- trace_id is generated at SO creation time (`Guid.NewGuid()`) and stored on `demand.SaleOrders.trace_id`
- When auto-PO is created from SO, the SAME trace_id is copied to `procurement.PurchaseOrders.trace_id`
- When Sales Invoice is created from SO, the SAME trace_id is copied to `finance.SalesInvoices.trace_id`
- DocumentTimeline writes always via `Hangfire BackgroundJob.Enqueue` — never in the main transaction
- Timeline events are APPENDED to the existing JSON array (never overwritten) via `JSON_MODIFY` or read-modify-write
- If a PO was already created from a Quotation (has a trace_id), and then a SO links to that PO, the SO's trace_id joins the same chain by cross-referencing
- The timeline UI shows events chronologically with document hyperlinks for navigation

13.7 Multi-Tenancy Enforcement

All timeline operations **MUST** enforce the existing multi-tenancy pattern:

- DocumentTimelines table includes `org_id` column with EF Core `HasQueryFilter(t => t.OrgId == _currentOrg)`
- Timeline queries always filter by `org_id` — a tenant can NEVER see another tenant's timeline events
- The trace_id is unique WITHIN an org (`IX_DT_TraceId` index is on `org_id + trace_id`)
- Timeline API endpoints extract `org_id` from JWT `org_id` claim — no client-supplied `org_id` accepted
- Cross-tenant trace_id collision is harmless: different `org_id` means different timeline row
- Hangfire timeline write jobs receive `org_id` as parameter — they do NOT rely on `HttpContext` (which is unavailable in background jobs)

```
// Hangfire job signature includes org_id explicitly:
BackgroundJob.Enqueue(() => _timelineService.AppendEvent(
    orgId: currentUser.OrgId, // captured at enqueue time
    traceId: saleOrder.trace_id,
    event: new TimelineEvent { ... }));
```

This follows the same multi-tenancy pattern used across all 14 schemas: auth, lookups, suppliers, inventory, demand, procurement, warehouse, logistics, finance, reports, workflow, hangfire, material, tenant.

Section 14 - Procurement Impact (Back-to-Back & Drop Ship)

14.1 Back-to-Back PO Details

Covered in detail in Section 6. Key additions to existing PO table:

- source: MANUAL | BACK_TO_BACK | DROP_SHIP | MIR
- linked_so_line_id, linked_so_id: FK references to the originating SO
- auto_generated_qty, auto_generated_price, auto_selected_supplier_id: Original values before user edits

14.2 Margin Calculation

```
margin = SO_line.unit_price - PO_line.unit_price  
margin_percent = (margin / SO_line.unit_price) * 100
```

Stored on SaleOrderLines as informational fields. Shown in Margin Analysis report (Section 15).

14.3 trace_id Propagation

All auto-created POs carry the originating SO's trace_id. See Section 13 for full trace_id / DocumentTimeline specification. The PO timeline shows:

- Originating Sale Order (linked via trace_id)
- Supplier selection method used (DEFAULT_SUPPLIER / BEST_MATCH)
- GRN receipt events with auto-reservation trigger
- Full chain navigation: SO → PO → GRN → Reservation → Fulfillment → Invoice

Section 15 - Sales Reports

#	Report	Description	Filters
R1	Sales Order Register	All SOs with status, totals, customer, delivery mode	Date range, status, customer, delivery_mode
R2	Customer Ledger Report	Receivables ledger per customer	Customer, date range
R3	Aging Receivables	Outstanding invoices by age bucket (0-30, 31-60, 61-90, 90+)	As-of date, customer
R4	Sales by Product	Revenue and quantity by product/variant	Date range, category, product
R5	Sales by Customer	Revenue, order count, avg order value per customer	Date range, customer
R6	Fulfillment Status	Open fulfillments, picking progress, delivery mode	Date range, warehouse, status, mode
R7	Margin Analysis	Selling price vs cost, margin % by product/customer/SO	Date range, product, customer
R8	Sales vs Purchase Comparison	Revenue vs COGS, gross margin by period	Date range, category
R9	Product Ledger Report	All movements per product with running qty and value	Product, variant, date range
R10	Product Profitability	Revenue - COGS per product, ranked by margin	Date range, category

All reports follow: GET /api/reports/sales/{name}?filters... with PDF (QuestPDF) and Excel export.

Section 16 - Implementation Order (30 Dev Days)

Step	Component	Depends On	Est. Days	Schema
1	BusinessPartner migration (rename + expanded types)	-	2	suppliers
2	Partner API + backward-compat aliases + carrier/service types	Step 1	2	suppliers
3	Default Supplier on ProductVariants + PricingRules	Step 1	2	inventory
4	SUPPLY_DEPT_ADMIN role + SaleOrderConfig	Step 1	1	auth + demand
5	SaleOrder + SaleOrderLines + availability check	Steps 1-4	3	demand
6	Extend IStockReservationService for SALES_ORDER source type	Step 5	1	inventory (shared)
7	Email Intimation Service + templates	Step 5	2	demand
8	Back-to-Back PO (auto-create, supplier selection, draft/approval) + trace_id propagation	Steps 5-6	3	procurement + workflow
9	PO-SO Timeline Linkage (DocumentTimeline events, timeline API)	Step 8	2	workflow
10	Drop Ship PO flow	Step 9	1	procurement
11	Extend delivery_orders for SALE_ORDER source + self-pickup + goods issue	Step 6	2	logistics
12	Sales Invoice + CustomerLedger	Step 11	2	finance
13	Customer Payment + Allocation	Step 12	2	finance
14	Product Ledger	Steps 11-12	2	finance
15	Sales Reports (10 reports)	Steps 5-14	3	reports

Total estimated effort: 30 development days (reduced from 32 by reusing existing logistics infrastructure)

16.1 Migration Script Order

```
dotnet ef migrations add RenameSupplierToBusinessPartner
dotnet ef migrations add AddPartnerTypeFlagsAndCarrierFields
dotnet ef migrations add AddDefaultSupplierToVariants
dotnet ef migrations add CreatePricingRules
dotnet ef migrations add CreateSupplyDeptAdminRole
dotnet ef migrations add CreateSaleOrderConfig
dotnet ef migrations add CreateSaleOrderTables
dotnet ef migrations add AddSalesOrderToReservationSourceType
dotnet ef migrations add CreateSaleOrderIntimations
dotnet ef migrations add AddPOSourceAndLinkedFields
dotnet ef migrations add AddSOTimelineEvents
dotnet ef migrations add ExtendDeliveryOrdersForSaleOrderSource
dotnet ef migrations add CreateSalesInvoiceTables
dotnet ef migrations add CreateCustomerLedger
dotnet ef migrations add CreateCustomerPaymentTables
dotnet ef migrations add CreateProductLedger
```

Section 17 - Acceptance Criteria & Test Scenarios

#	Scenario	Steps	Expected Result
TC-01	Vendor backward compat	1. GET /api/suppliers 2. Verify only is_vendor=1 returned 3. Create PO with existing vendor	All procurement flows unchanged
TC-02	Create CARRIER partner	1. POST /api/partners with is_carrier=1 2. Verify carrier appears in shipment carrier dropdown 3. Assign to fulfillment	Carrier created, usable in fulfillment
TC-03	Create SERVICE_PROVIDER	1. POST /api/partners with is_service_provider=1 2. Verify service_categories saved 3. Create service PO	Service provider works in procurement
TC-04	SO fully in-stock	1. Ensure 50 units available 2. Create SO for 30, confirm 3. Verify reservation, email sent	Reserved, email shows 'fully available'
TC-05	SO partial stock (SPLIT)	1. 60 units available, SO for 100 2. Confirm 3. Verify: 60 reserved, PO for 40 in DRAFT 4. Email shows both	Split handling, email accurate
TC-06	BEST_MATCH supplier selection	1. Config: supplier_selection_mode=BEST_MATCH 2. Multiple vendors with scorecards/rates 3. Confirm SO with deficit 4. Verify best-scored vendor selected	Composite score selects optimal vendor
TC-07	DRAFT_ONLY PO approval	1. Config: auto_po_approval_mode=DRAFT_ONLY 2. Trigger auto-PO 3. Verify PO in DRAFT 4. User edits qty/price 5. Manual submit	PO not sent until manual action
TC-08	Self-pickup flow	1. SO with delivery_mode=SELF_PICKUP 2. Confirm, create fulfillment 3. Pick, pack, READY_FOR_PICKUP 4. Record pickup person details 5. Status=PICKED_UP 6. Verify gate pass generated	No shipment, stock deducted via SALES_HANOVER
TC-09	Partial fulfillment (3 picks)	1. SO: 100 units 2. Fulfill 40 (shipped) 3. Fulfill 30 (shipped) 4. Fulfill 30 (self-pickup) 5. Verify SO=FULFILLED	Multiple fulfillments, mixed delivery modes
TC-10	Product Ledger entries	1. GRN 100 units at \$10 2. Sell 30 units at \$15 3. Verify PURCHASE entry, SALE entry 4. WAC = \$10, COGS = \$300, Revenue = \$450	Ledger accurate, WAC correct
TC-11	Email intimation content	1. SO with SPLIT: 50 in stock, 50 deficit 2. Drop ship on another line 3. Verify email contains: - Stock reservation details - PO creation notice - Drop ship notice	Email complete with all scenarios
TC-12	Multi-tenant isolation	1. Create customer in Org A 2. Login as Org B 3. Cannot access Org A customer 4. Cannot reference in SO	Complete isolation
TC-13	SO→PO trace_id propagation	1. Create SO, note trace_id 2. Confirm SO with deficit 3. Auto-PO created 4. Verify PO.trace_id = SO.trace_id 5. GET /api/sale-orders/{id}/timeline 6. Verify PO_CREATED_FROM_SO event in timeline	Same trace_id, events in DocumentTimelines
TC-14	Full chain timeline (SO→PO→GRN→Fulfillment→Invoice)	1. SO confirmed (SPLIT: 60 stock + 40 PO) 2. PO approved, GRN received 3. Fulfill all 100, deliver 4. Issue invoice 5. GET /api/timeline/{traceId}	All 7+ events in chronological order, all linked docs visible
TC-15	Timeline non-blocking write	1. Confirm SO that triggers auto-PO 2. Verify SO confirmation response < 2s 3. Timeline event appears within 5s (async)	Business transaction not slowed by timeline write

17.1 Non-Functional Requirements

- All new tables include org_id with HasQueryFilter for tenant isolation
- All list APIs support pagination (page, pageSize) with max 100 per page

- All writes wrapped in explicit transactions
- Ledger writes (Customer + Product) in SAME transaction as business action
- Timeline events via Hangfire BackgroundJob.Enqueue (non-blocking)
- Email sending via Hangfire (non-blocking, retry on failure)
- SaleOrderConfig changes require SUPPLY_DEPT_ADMIN role

17.2 Index Recommendations

Table	Index	Columns
BusinessPartners	IX_BP_OrgType	org_id, partner_type, is_active
BusinessPartners	IX_BP_OrgFlags	org_id, is_vendor, is_customer, is_carrier, is_service_provider
SaleOrders	IX_SO_OrgStatus	org_id, status, order_date
SaleOrders	IX_SO_Partner	org_id, partner_id
SaleOrderLines	IX_SOL_Variant	variant_id, status
StockReservations	IX_SR_Variant	org_id, variant_id, status
Fulfillments	IX_FUL_SO	sale_order_id, status
SalesInvoices	IX_SINV_Partner	org_id, partner_id, status
SalesInvoices	IX_SINV_DueDate	org_id, due_date, balance_due
CustomerLedger	IX_CL_Partner	org_id, partner_id, entry_date
ProductLedger	IX_PL_Variant	org_id, variant_id, entry_date
ProductLedger	IX_PL_Product	org_id, product_id, entry_date
PricingRules	IX_PR_Variant	org_id, variant_id, is_active, effective_from
SaleOrderIntimations	IX_SOI_SO	sale_order_id, event_type
DocumentTimelines	IX_DT_TraceId	org_id, trace_id (UNIQUE)
PurchaseOrders	IX_PO_TraceId	org_id, trace_id
PurchaseOrders	IX_PO_LinkedSO	linked_so_id, linked_so_line_id

17.3 Rollback Strategy

- Each migration is separate and revertible with dotnet ef database update <previous-migration>
- BusinessPartner rename can be rolled back by renaming table back to Suppliers
- All new tables can be dropped without affecting existing procurement or logistics data
- PO source/linked fields are nullable additions that can be dropped
- delivery_orders additions (sale_order_id, delivery_mode, pickup fields) are nullable and can be dropped
- SALES_ORDER added to ReservationSourceType enum — removal requires clearing sales reservations first
- SaleOrderConfig is independent and can be dropped
- ProductLedger is independent append-only table, safe to drop
- SO timeline events in DocumentTimelines can be removed by filtering JSON array (events are additive, no destructive impact)